

Projeto Final

Laboratório de Bases de Dados - SCC0241

Interface de controle e exploração de base de dados da Fórmula 1

Catarina Moreira Lima – 8957221
Amanda Kasat Baltor – 13727210
Vitor Hugo Almeida Couto – 13672787

Junho de 2026

1 Introdução

Este trabalho apresenta o desenvolvimento de um protótipo de aplicação completa para exploração da base de dados da Fórmula 1 integrada a dados geográficos de cidades, países e aeroportos.

A aplicação permite autenticar usuários com perfis distintos, apresentar *dashboards* personalizados, executar relatórios e realizar operações de cadastro conforme as permissões de cada tipo de usuário.

2 Preparação da Base de Dados

A base utilizada foi preparada a partir dos *scripts* desenvolvidos ao longo da disciplina. A execução foi organizada na seguinte ordem:

1. `create_table.sql`: criação do esquema relacional;
2. `insert_table.sql`: carga dos arquivos CSV e TSV;
3. `clean_data.sql`: normalização, deduplicação e ajuste de vínculos entre as tabelas;
4. `app_users.sql`: usuários, autenticação, auditoria e triggers;
5. `app_actions.sql`: ações de cadastro e consulta por perfil;
6. `app_indexes.sql`: índices auxiliares;
7. `app_views.sql`: views reutilizáveis pela aplicação;
8. `app_dashboard.sql`: funções de *dashboard*;
9. `app_reports.sql`: funções de relatórios.

2.1 Execução com Docker

Para facilitar a reprodução do ambiente entre os membros do grupo, foi utilizado Docker com PostgreSQL. A carga foi separada em dois scripts: um para a base de dados e outro para a camada da aplicação.

```
./scripts/load_database.sh
./scripts/load_app.sh
```

O script `load_database.sh` executa a criação do esquema, a carga dos dados e a limpeza. O script `load_app.sh` executa os arquivos de usuários, ações, índices, views, dashboards e relatórios. Assim, durante o desenvolvimento de novas funções, é possível recarregar apenas a camada da aplicação.

Os dados de conexão utilizados foram:

- Banco: `f1db`
- Usuário: `f1user`
- Porta: `5433`

3 Views Reutilizáveis

Foi criado o arquivo `app_views.sql` para concentrar consultas reutilizáveis pela aplicação. A view `vw_resultados_corridas` reúne os dados de resultados de corrida com temporada, piloto, escuderia, status e circuito.

Essa view centraliza junções frequentes entre as tabelas `results`, `races`, `seasons`, `drivers`, `constructors`, `status` e `circuits`. Com isso, as funções de *dashboard* e de relatórios ficam mais simples e reutilizam a mesma base lógica. Os filtros específicos continuam sendo aplicados nas funções, de acordo com o perfil do usuário logado, como administrador, escuderia ou piloto.

Também foi criada a view `vw_aeroportos_cidades_paises`, que reúne aeroportos, cidades, países e tipos de aeroporto. Essa view é utilizada no relatório de aeroportos próximos a uma cidade brasileira, mantendo na função apenas o filtro pela cidade informada e o cálculo da distância geográfica.

A view `vw_corridas_resumo` resume cada corrida com ano, rodada, circuito, voltas registradas e quantidade de pilotos participantes. Ela é usada no relatório hierárquico de corridas do administrador, evitando repetir a agregação dentro da função de relatório.

A view `vw_escuderia_pilotos` reúne escuderias, pilotos, países e a origem do vínculo entre escuderia e piloto. Essa view é construída sobre a tabela `app_escuderia_pilotos`, que combina vínculos históricos inferidos de `results` com vínculos criados pela importação de pilotos por arquivo.

4 Modelo de Usuários e Autenticação

A aplicação possui três tipos de usuários, cada um com permissões e visões distintas:

- **Admin:** usuário administrador do sistema;
- **Escuderia:** usuário associado a uma escuderia;

- **Piloto:** usuário associado a um piloto.

Foi criada a tabela `users`, contendo os atributos `userid`, `login`, `password`, `tipo` e `id_original`. O atributo `id_original` referencia o identificador original de piloto ou escuderia, conforme o tipo de usuário.

4.1 Proteção das Senhas

A autenticação foi implementada diretamente pela tabela `users`. Por isso, as senhas não são armazenadas em texto puro. Foi utilizada a extensão `pgcrypto`, com as funções `crypt()` e `gen_salt()`. A função `app_alterar_senha` permite trocar a senha validando a senha atual do usuário.

```
CREATE OR REPLACE FUNCTION app_hash_password(p_plain_password TEXT)
RETURNS TEXT AS $$
BEGIN
    RETURN crypt(p_plain_password, gen_salt('bf'));
END;
$$ LANGUAGE plpgsql;
```

4.2 Auditoria de Acesso

Foi criada a tabela `users_log`, responsável por registrar eventos de `LOGIN` e `LOGOUT`. A função `app_login` registra automaticamente a entrada do usuário quando a autenticação é realizada com sucesso.

5 Triggers Implementadas

5.1 Sincronização de Pilotos com Usuários

A trigger associada à tabela `drivers` cria automaticamente um usuário do tipo `Piloto` sempre que um novo piloto é cadastrado. Em atualizações, a trigger ajusta o login quando necessário, mas preserva a senha já alterada pelo usuário.

5.2 Remoção de Usuários de Pilotos

Também foi criada uma trigger associada à tabela `drivers` para remover automaticamente o usuário do tipo `Piloto` quando o piloto correspondente é removido. Antes da remoção do usuário, os registros relacionados em `users_log` são apagados para respeitar a chave estrangeira existente entre `users_log` e `users`.

5.3 Sincronização de Escuderias com Usuários

A trigger associada à tabela `constructors` cria automaticamente um usuário do tipo `Escuderia` sempre que uma nova escuderia é cadastrada. Em atualizações, a trigger ajusta o login quando necessário, mas preserva a senha já alterada pelo usuário.

5.4 Remoção de Usuários de Escuderias

Foi criada uma trigger associada à tabela `constructors` para remover automaticamente o usuário do tipo `Escuderia` quando a escuderia correspondente é removida. Assim como no caso dos pilotos, os registros associados em `users_log` são removidos antes da exclusão do usuário.

5.5 Atualização de Data em Usuários

A trigger de atualização da tabela `users` atualiza automaticamente o campo `updated_at` quando uma tupla é modificada.

5.6 Sincronização de Vínculos entre Escuderias e Pilotos

A trigger associada à tabela `results` mantém a tabela `app_escuderia_pilotos` atualizada quando novos resultados criam vínculos entre pilotos e escuderias. Essa estrutura aceita que um mesmo piloto esteja vinculado a mais de uma escuderia, preservando mudanças de escuderia e histórico de carreira.

6 Ações Implementadas

Foram criadas funções auxiliares para representar ações solicitadas na especificação:

- `app_admin_cadastrar_escuderia`: cadastra uma nova escuderia e deixa a criação do usuário correspondente a cargo da trigger;
- `app_admin_cadastrar_piloto`: cadastra um novo piloto e deixa a criação do usuário correspondente a cargo da trigger;
- `app_escuderia_processar_importacao_pilotos`: processa linhas carregadas por arquivo em uma tabela de apoio, cadastrando pilotos novos e criando vínculos com a escuderia;
- `app_escuderia_listar_pilotos`: lista os pilotos vinculados à escuderia no cadastro da aplicação;
- `app_escuderia_consultar_piloto_por_sobrenome`: consulta pilotos pelo sobrenome que *já correram* pela escuderia logada, consultando diretamente a tabela `results`, conforme a dica do enunciado.

Para suportar a importação de pilotos por escuderia, foi criada uma tabela própria de vínculo entre escuderias e pilotos. A relação original entre piloto e escuderia aparece apenas em `results`; portanto, pilotos recém-importados ainda não teriam vínculo visível caso a aplicação dependesse somente dos resultados históricos. Por isso, a tabela é inicialmente populada com pares distintos de `results`, recebe vínculos criados por importação de arquivo e é atualizada por trigger quando novos resultados são registrados.

Após a revisão das funções, foi mantida a separação entre consultas de desempenho e consultas cadastrais. Dashboards e relatórios que calculam pontos, vitórias, temporadas, corridas ou status usam `results` ou `vw_resultados_corridas`, pois dependem de participação real em corridas. A consulta de piloto por sobrenome também se baseia

em `results`, pois o enunciado pede pilotos que *já correram* pela escuderia. Já a *listagem* cadastral de pilotos vinculados à escuderia (`app_escuderia_listar_pilotos`) usa `app_escuderia_pilotos` ou `vw_escuderia_pilotos`, pois nesse caso faz sentido incluir também pilotos importados por arquivo, mesmo antes de existirem resultados para eles.

7 Dashboards

7.1 Dashboard do Administrador

O dashboard do administrador apresenta:

- quantidade total de pilotos, escuderias e temporadas;
- corridas cadastradas na temporada mais recente;
- escuderias da temporada mais recente com total de pontos;
- pilotos da temporada mais recente com total de pontos.

7.2 Dashboard da Escuderia

O dashboard da escuderia apresenta:

- quantidade de vitórias da escuderia;
- quantidade de pilotos diferentes que já correram pela escuderia;
- primeiro e último ano com dados da escuderia.

7.3 Dashboard do Piloto

O dashboard do piloto apresenta:

- primeiro e último ano com dados do piloto;
- desempenho por ano e circuito, incluindo pontos, vitórias e corridas disputadas.

8 Relatórios

8.1 Relatório 1 – Admin: Resultados por Status

Este relatório apresenta a quantidade de resultados agrupados por status de corrida.

8.2 Relatório 2 – Admin: Aeroportos Próximos a Cidade Brasileira

Este relatório recebe o nome de uma cidade e retorna aeroportos brasileiros dos tipos `medium_airport` e `large_airport` localizados a até 100 km da cidade pesquisada. Os dados de aeroporto são obtidos pela view `vw_aeroportos_cidades_paises`.

Para o cálculo de distância foi utilizada a extensão `earthdistance`, juntamente com `cube`.

```
earth_distance(  
    ll_to_earth(c.latitude, c.longitude),  
    ll_to_earth(vap.latitude_aeroporto, vap.longitude_aeroporto)  
) / 1000.0
```

A busca pelo nome da cidade é *insensível a acentos*: utilizamos a extensão `unaccent` no filtro (`unaccent(lower(c.name)) = unaccent(lower(p_city_name))`), de modo que “sao paulo”, “São Paulo” e “SÃO PAULO” retornam o mesmo resultado. Como consequência dessa escolha, o índice `idx_cities_country_lower_name_coordinates` passa a ter papel secundário nesse filtro específico (a expressão indexada não corresponde mais exatamente ao predicado); ele continua útil para restringir cidades por país e por coordenadas disponíveis.

8.3 Relatório 3 – Admin: Relatório Hierárquico de Corridas

O relatório 3 foi dividido em duas partes:

- `app_admin_relatorio_3_escuderias_pilotos`: lista todas as escuderias cadastradas e a quantidade de pilotos que correram por cada uma;
- `app_admin_relatorio_3_hierarquico_corridas`: organiza as corridas em formato hierárquico.

A parte hierárquica foi organizada em três níveis:

1. total de corridas cadastradas;
2. estatísticas de corridas por circuito;
3. detalhes das corridas de cada circuito.

Para representar o relatório multinível em formato tabular, a função retorna uma coluna `nivel`. Assim, as linhas de nível 1 representam o total geral, as linhas de nível 2 representam o resumo por circuito e as linhas de nível 3 representam as corridas pertencentes a cada circuito.

8.4 Relatório 4 – Escuderia: Vitórias por Piloto

Este relatório lista os pilotos que correram por uma escuderia e a quantidade de vezes em que cada piloto obteve primeira posição.

8.5 Relatório 5 – Escuderia: Resultados por Status

Este relatório apresenta a quantidade de resultados por status, limitada ao escopo da escuderia logada.

8.6 Relatório 6 – Piloto: Pontos por Ano e Corridas

Este relatório apresenta os pontos obtidos pelo piloto por ano de participação, indicando as corridas em que os pontos foram obtidos.

O total de pontos por ano é retornado como `NUMERIC(10,2)`, e não como inteiro. Isso porque a Fórmula 1 atribui meios-pontos em situações específicas (por exemplo, corridas encurtadas). Essa escolha mantém o relatório consistente com o *dashboard* do piloto, que também trabalha com pontos fracionários.

8.7 Relatório 7 – Piloto: Status das Corridas

Este relatório apresenta a quantidade de resultados por status nas corridas em que o piloto participou.

9 Índices Criados

Os índices foram criados para auxiliar consultas com filtros, junções e ordenações frequentes.

Índice	Justificativa
<code>idx_results_status</code>	Facilitar consultas que agrupam ou filtram por status de resultado.
<code>idx_results_constructor_driver</code>	Auxiliar consultas que agrupam resultados por escuderia e contam pilotos distintos, especialmente no Relatório Admin 3 e nos relatórios de escuderia.
<code>idx_cities_country_lower_name_coordinates</code>	Auxiliar a busca de cidades brasileiras por nome no Relatório 2, considerando apenas cidades com coordenadas disponíveis.
<code>idx_airports_city_type_coordinates</code>	Auxiliar a seleção de aeroportos com cidade, tipo e coordenadas disponíveis no Relatório 2.
<code>idx_results_driver_race_points_positive</code>	Auxiliar o Relatório 6, filtrando resultados pontuados por piloto e facilitando a junção com corridas.
<code>idx_app_escuderia_pilotos_piloto</code>	Auxiliar consultas que partem de um piloto para encontrar vínculos com escuderias na camada da aplicação.
<code>idx_app_import_pilotos_escuderia_pendentes</code>	Auxiliar o processamento de linhas pendentes carregadas por arquivo para uma escuderia.
<code>idx_drivers_lower_family_name</code>	Auxiliar buscas de pilotos por sobrenome sem diferenciar letras maiúsculas e minúsculas.

10 Interface da Aplicação

A interface foi desenvolvida em Streamlit (Python) e roda localmente, conectando-se ao PostgreSQL em execução no container Docker. Toda consulta feita pela aplicação chama, de forma explícita, uma *view* ou função já criada no banco; não há lógica de negócio duplicada em Python nem uso de *ORM*, o que mantém o SQL visível para avaliação.

A aplicação é centrada em três telas principais — login, *dashboard* e relatórios —, cada uma com variações de acordo com o tipo de usuário autenticado. Após o login, a navegação entre as telas é feita por um menu na barra lateral, organizado em itens de texto clicáveis e adaptado ao perfil do usuário: o administrador acessa “Visão Geral”, “Cadastros” e “Relatórios”; a escuderia acessa “Visão Geral”, “Ações” e “Relatórios”; e o piloto, por ser somente leitura, acessa apenas “Visão Geral” e “Relatórios”. A mesma barra lateral exibe a logo da Fórmula 1, o nome do usuário logado e o botão de *logout*, disponível em qualquer tela.

10.1 Tela de Login

A tela inicial solicita login e senha e autentica o usuário pela função `app_login`, que valida a senha protegida com `pgcrypto` e registra o evento de LOGIN na tabela `users_log`. Em caso de credenciais inválidas, uma mensagem de erro é exibida; com a autenticação bem-sucedida, o usuário é direcionado à tela de *dashboard*.

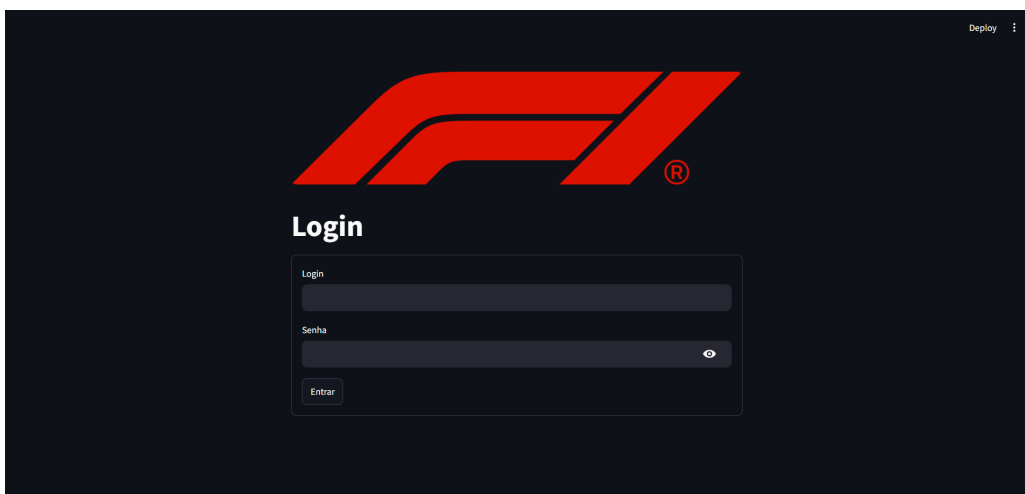


Figura 1: Tela de login da aplicação.

10.2 Tela de Dashboard

A página “Visão Geral” apresenta as informações sumarizadas do perfil logado. O administrador vê os totais de pilotos, escuderias e temporadas, as corridas da temporada mais recente e os pontos por escuderia e por piloto, com gráficos de apoio. A escuderia vê o número de pilotos, as vitórias e o primeiro e último ano com dados. O piloto vê a escuderia atual, seu nome completo e o desempenho por ano e circuito. Os gráficos foram construídos com Altair em modo estático (sem *zoom* ou arraste), priorizando a leitura dos dados.

As ações de cada perfil ficam em página própria do menu: “Cadastros”, para o administrador (cadastrar escuderia e piloto), e “Ações”, para a escuderia (consultar piloto por

sobrenome e importar pilotos por arquivo). O piloto não possui ações de cadastro.

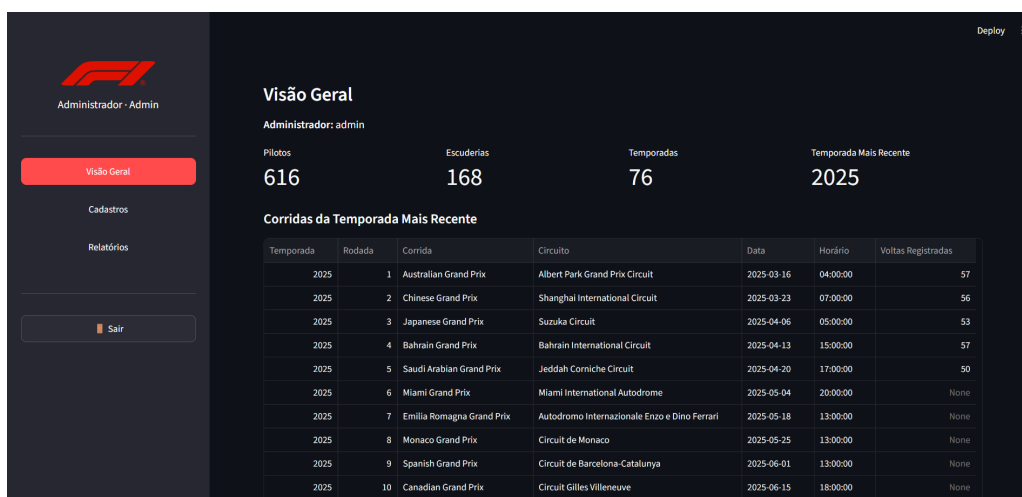


Figura 2: Tela de dashboard (Visão Geral).

10.3 Tela de Relatórios

A página de relatórios lista apenas os relatórios disponíveis ao perfil logado. Ao escolher um relatório, a aplicação executa a função correspondente e exibe o resultado; em seguida, é possível retornar à lista. O Relatório 3, hierárquico, é apresentado como um *drill-down*: cada circuito pode ser expandido para mostrar suas corridas.



Figura 3: Tela de relatórios.

11 Decisões de Projeto

Esta seção reúne as principais escolhas do grupo e a justificativa de cada uma.

11.1 Modelagem e tratamento de dados (Trabalho Prático T1)

A modelagem do esquema e o tratamento/limpeza dos dados utilizados nesta aplicação são os disponibilizados no *gabarito de referência* do Trabalho Prático T1. Optamos por

reaproveitar essa versão de referência por ser a base já validada ao longo da disciplina, sobre a qual a aplicação foi construída.

11.2 Escolha do Streamlit como tecnologia de interface

Escolhemos o **Streamlit** (Python) por permitir construir as telas, formulários e, principalmente, os *dashboards* de forma prática e rápida, com componentes prontos de tabela, métrica e gráfico, sem exigir um *front-end* web separado. Assim, o esforço ficou concentrado no que o trabalho de fato avalia — consultas, funções, views, índices e *triggers* —, enquanto a camada de interface apenas chama as views e funções já criadas no banco. Os gráficos dos *dashboards* foram montados com Altair (já incluído no Streamlit), configurados de forma estática (sem zoom ou arraste) para priorizar a leitura clara dos dados.

11.3 Autenticação pela tabela `users` com hash, em vez de usuários reais do SGBD

O enunciado permite duas abordagens: usar usuários reais do PostgreSQL (com autenticação **SCRAM-SHA-256**) ou autenticar diretamente por uma tabela `users`, desde que a senha não seja armazenada em texto puro. Optamos pela segunda. Assim, não é necessário criar e gerenciar centenas de *roles* reais no SGBD (uma por piloto e por escuderia), o protótipo fica mais portátil entre máquinas e a lógica de login/logout pode ser controlada pela própria aplicação. As senhas são armazenadas como *hash* gerado com `pgcrypto` (`crypt()` com `gen_salt('bf')`), isto é, *bcrypt*), nunca em texto puro.

11.4 Lógica de negócio no banco e SQL explícito na aplicação

Todas as operações da aplicação (consultas, cadastros, importação, login) são implementadas como *views* e *funções* no banco. A camada em Python apenas chama essas *views*/funções com SQL explícito, sem utilizar um ORM. Essa decisão concentra a regra de negócio em um único lugar (o banco), evitando duplicação de lógica entre SQL e Python.

11.5 Tabela própria de vínculo entre escuderia e piloto

Na base original, a associação entre piloto e escuderia só existe na tabela `results`, ou seja, apenas após participação real em corrida. Como a importação de pilotos por arquivo pode cadastrar pilotos que ainda não correram, criamos a tabela `app_escuderia_pilotos` para registrar esses vínculos. Ela é populada inicialmente com os pares distintos de `results`, recebe os vínculos criados na importação e é mantida atualizada por *trigger* quando novos resultados surgem. A tabela aceita um mesmo piloto vinculado a mais de uma escuderia, preservando mudanças de equipe ao longo da carreira.

11.6 Separação entre consultas de desempenho e consultas cadastrais

Como consequência da decisão anterior, separamos dois tipos de consulta. As consultas de *desempenho* (pontos, vitórias, temporadas, status) usam `results` ou `vw_resultados_corridas`, pois dependem de participação real em corrida. A *consulta de piloto por sobrenome* também usa `results`, porque o enunciado pede pilotos que *já correram* pela escuderia

(e dá a dica de consultar `results`). Já a *listagem* cadastral de pilotos vinculados usa `app_escuderia_pilotos`, pois aí faz sentido incluir também os pilotos recém contratados que ainda não têm resultados.

11.7 Sincronização e consistência de usuários por *triggers*

Optamos por delegar a criação e a remoção dos usuários de piloto e escuderia a *triggers* nas tabelas `drivers` e `constructors`, em vez de fazer isso no código da aplicação. Assim, qualquer inserção nessas tabelas — venha da interface ou de um comando SQL direto — gera o usuário correspondente automaticamente. Quando o *login* gerado já existe para outro usuário, a *trigger* cancela a operação com `RAISE EXCEPTION`, impedindo inserção inconsistente. Na prática, como `constructor_ref` e `driver_ref` são únicos, a restrição `UNIQUE` da tabela de origem costuma barrar o caso comum de referência repetida antes mesmo da *trigger*; a verificação na *trigger* garante o cancelamento também no cenário em que o *login* já existe por outro caminho.

11.8 Busca de cidade insensível a acentos (Relatório 2)

Para melhorar a usabilidade, a busca de cidade do Relatório 2 ignora acentuação e caixa, usando a extensão `unaccent`. Dessa forma, “sao paulo” e “São Paulo” retornam o mesmo resultado. Como consequência, o índice de cidade deixa de casar exatamente com o predicado, passando a ter papel secundário nesse filtro.

11.9 Pontuação com casas decimais (Relatório 6)

O total de pontos por ano é tratado como `NUMERIC(10,2)`, e não como inteiro, porque a Fórmula 1 atribui meios-pontos em situações específicas. Truncar para inteiro perderia esses 0,5 ponto e tornaria o relatório inconsistente com o *dashboard* do piloto.

11.10 Controle de acesso por perfil

Cada tipo de usuário enxerga apenas o que lhe cabe. As funções de escuderia e de piloto recebem o `id_original` do usuário logado e filtram os dados por esse identificador. Na interface, o menu lateral exhibe apenas as páginas e ações do perfil autenticado, e o usuário do tipo Piloto é somente leitura, sem qualquer ação de cadastro, conforme o enunciado.

11.11 Importação de pilotos por *upload* de arquivo

Optamos por implementar a importação como *upload* pelo navegador (*widget* de envio de arquivo do Streamlit), o que é mais prático de demonstrar e não depende de o arquivo estar em um caminho específico do servidor. O arquivo é carregado em uma tabela de *staging* e processado pela função `app_escuderia_processar_importacao_pilotos`, que rejeita linhas cujo nome e sobrenome já existam em outro piloto.

11.12 Organização da carga em dois *scripts*

A carga foi separada em `load_database.sh` (esquema, dados e limpeza) e `load_app.sh` (usuários, ações, índices, views, dashboards e relatórios). Isso permite recarregar apenas

a camada da aplicação durante o desenvolvimento, sem reprocessar toda a base a cada alteração em uma função, view ou índice.

12 Dificuldades Encontradas

De modo geral, o projeto não apresentou grandes dificuldades. As atividades realizadas ao longo do semestre foram muito úteis e puderam ser aproveitadas diretamente no trabalho final: várias das consultas, assim como agregações, filtros, criação de funções, *views* e *triggers*, além do cálculo de distância geográfica já haviam sido exercitadas em atividades anteriores. Isso reduziu bastante o esforço com as buscas do projeto, pois boa parte do raciocínio (e até de trechos de consulta) pôde ser reaproveitada.

Ainda assim, alguns pontos exigiram mais atenção, como costuma ocorrer em projetos desse tipo:

- integração de bases heterogêneas (Fórmula 1, países, cidades e aeroportos), que exige cuidado com chaves, vínculos e padronização de nomes;
- garantir que cada perfil de usuário acesse apenas as informações que lhe são permitidas;
- ajustes de ambiente e configuração (Docker e conexão com o banco) para que o projeto rodasse de forma reproduzível entre os integrantes do grupo.

Esses pontos foram tratados de forma incremental e, por já terem sido abordados de alguma maneira durante a disciplina, foram mais tranquilos de resolver nesse momento.

13 Conclusão

O trabalho atingiu o objetivo proposto: desenvolver um protótipo de aplicação completa para exploração da base de dados da Fórmula 1, integrada a dados geográficos de países, cidades e aeroportos. A ferramenta autentica três perfis de usuário (administrador, escuderia e piloto), apresenta *dashboards* personalizados, executa os relatórios solicitados e oferece as ações de cadastro, consulta e importação previstas no enunciado, sempre respeitando as permissões de cada perfil.

Do ponto de vista de banco de dados, o projeto reuniu os principais conceitos estudados na disciplina: criação e normalização do esquema, limpeza e deduplicação dos dados, consultas com junções, agregações e filtros, *views* reutilizáveis, funções e procedimentos armazenados, *triggers* para manter a consistência entre as tabelas, índices para apoiar as consultas mais frequentes e um modelo de autenticação com senhas protegidas. A opção por concentrar a lógica no banco e manter o SQL explícito na aplicação reforçou esses conceitos.

A integração entre os recursos do PostgreSQL e a interface em Streamlit mostrou-se eficaz para transformar consultas em informações organizadas e de fácil leitura para o usuário final. Como evolução futura, o projeto poderia incluir novos relatórios e filtros, gráficos comparativos entre temporadas, atualização da base com as corridas mais recentes e refinamentos adicionais na deduplicação de cidades feitas manualmente.